

面试手册

计算机网络

把基础，读成底气。

1 个科目 / 20 道问题

回答要点 • 常见追问 • 易错点 • 参考来源

整理日期：2026-10-09

本手册总结经典面试知识点，不是全网公司面经的穷举，也不代表真实招聘频次统计。题解为独立中文整理，具体版本与实现请参考题目中的原始资料。

C++ 以 C++17/20 为主；MySQL 以 8.4 LTS InnoDB 为主；Redis 以开源版常用功能为主。目录、PDF 书签及参考链接均可点击。

阅读目录

阅读目录	2
计算机网络	3
01. OSI 七层与 TCP/IP 模型怎么对应?	3
02. TCP 与 UDP 的区别和使用场景?	3
03. TCP 为什么通常需要三次握手?	4
04. TCP 关闭为什么常见四次挥手?	5
05. TIME_WAIT 为什么存在? 大量出现怎么办?	6
06. TCP 如何检测丢包并重传?	6
07. 流量控制、滑动窗口与零窗口探测?	7
08. 拥塞控制与流量控制有什么区别?	8
09. TCP 粘包与拆包如何解决?	8
10. HTTP/1.1 的连接复用和队头阻塞?	9
11. HTTP/2 怎样多路复用? 还会队头阻塞吗?	10
12. HTTP/3、QUIC 与 UDP 是什么关系?	10
13. HTTP 方法、幂等性和状态码怎么回答?	11
14. HTTP 强缓存与协商缓存如何工作?	12
15. HTTPS 与 TLS 1.3 握手如何理解?	12
16. DNS 解析过程和缓存、UDP/TCP 的关系?	13
17. IP 地址、子网掩码与 CIDR 如何计算?	14
18. 路由、最长前缀匹配和 NAT 的区别?	15
19. ARP 如何工作? IPv6 用什么?	15
20. 输入 URL 后发生什么? socket 在哪里?	16

计算机网络

从可靠传输到 Web 协议

覆盖 TCP/IP 与 Web 通信的高频考点。TCP 以 RFC 9293 为基础，HTTP 使用 9110 系列，明确区分 TLS 1.3、HTTP/2 与 HTTP/3。

01. OSI 七层与 TCP/IP 模型怎么对应？

网络基础 / 基础

OSI 是分层参考模型，TCP/IP 是实际互联网协议体系。可把应用、表示、会话功能归入应用层，再分传输、网际和链路层；部分教材将链路拆成数据链路与物理两层。

回答要点

1. 应用层处理 HTTP、DNS 等业务语义，传输层通过 TCP 或 UDP 为进程通信提供端口和传输能力，IP 负责跨网络寻址与转发，链路层处理同一链路内的帧传递。
2. 发送侧逐层封装首部，接收侧反向解析；路由器主要依据网络层信息转发，交换机常依据链路地址转发。实际设备也可能实现更高层功能，不能仅凭设备名称限定层级。
3. 分层是一种功能组织方式，现实协议可能跨越传统边界，例如 QUIC 在 UDP 上实现可靠流和安全握手；描述它时应说明其作用，而不强行套入唯一的 OSI 层号。

常见追问

追问：端口、IP 和 MAC 分别解决什么问题？答：端口帮助区分主机上的传输端点，IP 用于网络间寻址与路由，MAC 用于特定链路内交付。跨路由时链路首部常改变，端到端地址也可能受 NAT 影响。

易错点

易错点：认为 OSI 七层就是互联网每个报文必须严格走过的七个独立模块，或把 HTTP 的所有功能固定解释成会话层。参考模型和协议实现应区分。

延伸阅读 / 原始参考资料

RFC 1122 • Internet Host Communication Layers

MDN • HTTP Overview

02. TCP 与 UDP 的区别和使用场景？

TCP 与 UDP / 基础

TCP 提供面向连接、有序可靠的字节流，并包含流量和拥塞控制；UDP 提供保留消息边界的数据报，不内置可靠有序传输。选择取决于应用是否需要时效、顺序与自定义传输策略。

回答要点

1. TCP 负责排序、丢失恢复和重复处理，但连接中断时仍可能出现结果不确定，不能替业务提供事务成功保证；应用还需约定消息格式、超时以及必要的幂等或去重机制。
2. UDP 每次发送对应数据报边界，但网络仍可能丢失、乱序、重复，较大报文还可能面临分片问题；应用应控制消息大小，并在需要时实现反馈、重试和拥塞响应。
3. 实时媒体可优先考虑过期数据无需重传的策略，可靠文件传输常选择 TCP；QUIC 则在 UDP 上实现可靠传输，说明 UDP 可作为更高层可靠协议的承载，而非只能用于不可靠业务。

常见追问

追问：UDP 为什么不保证比 TCP 快？答：它减少部分协议机制，但实际延迟取决于链路、拥塞、重试和应用处理。若应用自己补齐可靠性，也会产生开销；应比较符合业务语义的完整实现及测量结果。

易错点

易错点：TCP 可靠不等于业务绝不失败，UDP 无握手不等于应用没有会话，也不等于其上层协议不能实现可靠传输。可靠性保证必须说明所在层级。

延伸阅读 / 原始参考资料

RFC 768 • User Datagram Protocol

Linux 手册 • tcp(7)

RFC 9000 • QUIC Transport

03. TCP 为什么通常需要三次握手？

TCP 与 UDP / 基础

常规握手依次交换 SYN、SYN+ACK、ACK，使双方同步各自初始序列号并确认对方已收到自己的建连信息。第三步避免服务器仅因旧 SYN 就把连接视为已完全建立。

回答要点

1. 客户端发送 SYN 并携带初始序列号 x ，服务器用 SYN+ACK 确认 $x+1$ ，同时公布自己的序列号 y ；客户端再确认 $y+1$ 。SYN 消耗一个序列号空间，方便可靠确认。
2. 三次交互让双方分别获知对端的发送初始状态和确认信息；若只用两次，服务器可能无法确认客户端是否收到其 SYN，也难以正确区分某些旧连接请求带来的情况。
3. 常规流程不是所有连接建立情况的唯一报文排列，同时打开、TCP Fast Open 和重传会呈现不同交互；回答应先使用 RFC 9293 的基础模型，再说明扩展与异常路径。

$C \rightarrow S$: SYN, seq= x

$S \rightarrow C$: SYN+ACK, seq= y , ack= $x+1$

$C \rightarrow S$: ACK, ack= $y+1$

常见追问

追问：第三次 ACK 丢失怎么办？答：服务器仍可能处在等待确认状态并重传 SYN+ACK；客户端收到后再次确认。客户端后续携带正确确认号的数据也可能推进状态，但不能据此假设第三步从来不重要。

易错点

易错点：只背诵双方确认能收能发，却不解释序列号同步与旧报文；或者把三次握手说成必须恰好三个网络数据包，忽略重传和特殊建连流程。

延伸阅读 / 原始参考资料

RFC 9293 • TCP — Connection Establishment

04. TCP 关闭为什么常见四次挥手？

TCP 与 UDP / 基础

TCP 是全双工，两方向的数据流需要分别终止。某端发送 FIN 表示该方向不再发送，另一端确认后仍可继续发送，待其结束再发 FIN；确认与 FIN 有时可以合并。

回答要点

1. 主动关闭方发送 FIN 后等待确认，对端确认只说明收到这一方向的终止声明，不意味着它也准备终止发送；这种半关闭允许一端结束请求后继续接收剩余响应数据。
2. 对端最终发送 FIN，主动方确认并进入相应结束状态；FIN 与 SYN 一样消耗序列号。正常有序关闭、异常 RST 和应用超时触发的关闭，其状态与数据保证不同。
3. 若对端已经没有待发送数据，它的 ACK 与 FIN 可放在同一报文中；同时关闭也有其他状态迁移，所以四次挥手是常见教学流程，而不是不可变化的报文数量定律。

常见追问

追问：CLOSE_WAIT 很多通常看什么？答：它常说明对端已结束发送，而本地应用尚未完成关闭，应该检查读到 EOF 后的处理、资源生命周期和异常路径；状态本身不是证据，仍需结合连接持续时间与业务。

易错点

易错点：对端 ACK 了 FIN 就等于两个方向同时关闭，或者所有关闭都走四个独立报文。半关闭、合并报文和 RST 需要分别解释。

延伸阅读 / 原始参考资料

Linux 手册 • shutdown(2)

Linux 手册 • close(2)

RFC 9293 • TCP — Connection Termination

05. TIME_WAIT 为什么存在？大量出现怎么办？

TCP 与 UDP / 进阶

TIME_WAIT 为已关闭连接保留一段状态，帮助处理对端 FIN 的重传并避免旧报文影响后续同标识连接。典型主动关闭方进入该状态，但同时关闭等情况需按状态机分析。

回答要点

1. 最后确认可能在网络中丢失，对端会重传 FIN；保留状态的一方能再次确认，帮助对端完成关闭。若立即销毁全部状态，可能无法按正常关闭语义响应这些重传。
2. 等待窗口还有隔离旧连接报文的作用，教学通常用 2MSL 表示；MSL 是最大报文生存期模型参数，真实系统的等待时长和复用策略依实现而异，不应固定背一个秒数。
3. 大量短连接和主动关闭会积累 TIME_WAIT，可优先使用连接复用、调整请求模式并排查临时端口压力；是否需要参数调优应看实际耗尽症状，而不是仅凭数量多就关闭保护。

常见追问

追问：TIME_WAIT 一定只在客户端吗？答：不是。角色名称与关闭方向无必然关系，服务器也可主动关闭而进入 TIME_WAIT，同时关闭时双方也可能经历该状态；判断要依据实际 FIN 和状态转换。

易错点

易错点：认为 TIME_WAIT 是内存泄漏，或可无代价强制删除。盲目复用连接标识可能破坏旧报文隔离，应明确系统版本、连接生命周期与真实瓶颈。

延伸阅读 / 原始参考资料

RFC 1337 • TIME-WAIT Assassination Hazards

Linux 手册 • tcp(7)

06. TCP 如何检测丢包并重传？

TCP 与 UDP / 进阶

TCP 依据确认和定时机制恢复丢失数据。超时重传采用随 RTT 更新的 RTO，重复确认可触发快速重传，SACK 提供已收到的数据范围；它们处理信息不足的不同情况。

回答要点

1. 发送方维护尚未被确认的序列范围，确认号通常表示下一个期望字节；没有及时收到确认可能因数据丢失，也可能因确认丢失或网络延迟，所以超时不能直接证明哪个报文丢失。
2. RTO 结合平滑 RTT 和波动估计，而非简单固定为一次 RTT；超时后需要退避。重传报文的 RTT 采样存在歧义，标准算法对这种样本有专门处理，避免错误缩短定时器。

3. SACK 让接收方描述不连续但已收到的范围，帮助发送方只恢复缺失片段；经典快速重传参考重复 ACK，现代实现还可能使用其他丢失检测机制，不能视为永远固定的一套规则。

常见追问

追问：收到三个重复 ACK 一定丢包吗？答：不一定，报文重排序也可能产生重复确认；算法根据线索推断丢失，并存在误判和恢复措施。解释时应区分经典规则与具体系统实际采用的检测算法。

易错点

易错点：把 ACK 数量直接等同于丢包数量，或认为超时重传固定等待一秒。RTT、抖动、退避和选择性确认会影响实际重传时机与范围。

延伸阅读 / 原始参考资料

RFC 6298 • Computing TCP's Retransmission Timer

RFC 2018 • TCP Selective Acknowledgment

RFC 5681 • TCP Congestion Control

07. 流量控制、滑动窗口与零窗口探测？

TCP 与 UDP / 进阶

流量控制避免发送方压垮接收方缓冲区。接收端通告可接收窗口 `rwnd`，发送方据此限制未确认数据；零窗口不表示连接断开，需要探测窗口是否重新开放。

回答要点

1. 滑动窗口让发送方无需每发一段就停下来等待，已确认数据腾出发送范围，未确认数据继续受窗口限制；窗口单位是序列空间中的字节，不能简单看成固定个数的数据包。
2. 接收应用读取慢会占满接收缓冲，通告窗口可能降到零；发送方暂缓正常发送，并通过持续探测避免窗口更新丢失后永久等待，应用仍应设置合适的业务超时。
3. TCP 首部基础窗口字段位宽有限，窗口扩大选项可支持更大的接收窗口；该能力在握手时协商。实际允许发送的数据还要同时受拥塞窗口和已在途数据量约束。

常见追问

追问：扩大接收缓冲一定增加吞吐吗？答：当瓶颈是窗口不足且链路带宽时延积较大时可能有用；若瓶颈在拥塞、应用处理、CPU 或磁盘，更大缓冲可能仅增加排队延迟，需要结合整个路径测量。

易错点

易错点：把 `rwnd` 与 `cwnd` 混成一个窗口，或认为接收窗口为零后只能断连。流量控制反映接收能力，拥塞控制反映网络路径允许的发送量。

延伸阅读 / 原始参考资料

RFC 7323 • TCP Extensions for High Performance

08. 拥塞控制与流量控制有什么区别？

TCP 与 UDP / 进阶

流量控制保护接收端，拥塞控制限制网络中过量在途数据。经典 TCP 通过窗口和阈值组织慢启动、拥塞避免及恢复；实际系统也可能采用 CUBIC、BBR 等算法。

回答要点

1. 发送量通常受 $\min(\text{cwnd}, \text{rwnd})$ 与在途数据共同约束；cwnd 是发送方根据网络反馈维护的状态，不是接收方直接通告，也不能通过扩大接收窗口就完全绕过。
2. 经典慢启动在确认驱动下快速扩大窗口，接近阈值后采用更谨慎的增长；发生超时或明确拥塞线索后降低发送量。具体增长、恢复和阈值更新规则依算法与事件不同。
3. 拥塞线索可以包括丢失、时延和 ECN 标记，不同算法关注的信号不同；讲解 Reno 风格规则时应指出这是经典模型，不将其行为直接套到所有操作系统和网络连接。

常见追问

追问：网络没有丢包是否表示不拥塞？答：不一定。队列可能已经增长而尚未溢出，表现为延迟上升；支持 ECN 的路径也可以标记拥塞。应同时观察吞吐、RTT、队列和显式反馈，而非只看丢包。

易错点

易错点：把慢启动说成增长最慢，或把拥塞避免描述成永远固定的线性增长。阶段名称、经典算法模型与实际部署算法是三个需要说明的层面。

延伸阅读 / 原始参考资料

RFC 5681 • TCP Congestion Control

RFC 9438 • CUBIC

09. TCP 粘包与拆包如何解决？

TCP 与 UDP / 基础

TCP 提供字节流，不保留应用 send 调用的消息边界。一次接收可能包含半条、多条或若干条消息的片段；所谓粘包与拆包是应用消息解析问题，需要明确封装协议。

回答要点

1. 长度前缀先给出载荷长度，接收方缓冲到完整消息再解析；必须限制最大长度并处理整数溢出、非法字段和提前断连，避免攻击者利用长度制造巨大分配或无限等待。

2. 分隔符适合文本协议，但载荷若包含分隔符需转义或有明确语法；固定长度简单但灵活性有限。选择取决于数据类型、吞吐、调试需求和协议可演进性。

3. `read` 或 `recv` 的返回值才是本次实际读取长度，不能假设一次请求会全部满足；发送端也要处理部分写入。接收解析器应保留剩余字节，持续读取并输出完整消息。

消息格式：4 字节长度（网络字节序） + N 字节载荷

接收循环：积累字节 → 校验长度 → 提取完整消息 → 保留余量

常见追问

追问：关闭 Nagle 能解决粘包吗？答：不能。它影响小块发送的聚合策略，字节流语义仍然不保留消息边界。即使每次写入立刻发送，网络分段、缓冲和读取行为也会使接收边界发生变化。

易错点

易错点：用 `sleep` 强行分隔消息，或按数据包长度直接推断业务消息长度。这些做法依赖偶然时序，真正可靠的边界来自应用协议与增量解析。

延伸阅读 / 原始参考资料

Linux 手册 • `recv(2)`

Linux 手册 • `send(2)`

Linux 手册 • `tcp(7)`

10. HTTP/1.1 的连接复用和队头阻塞？

HTTP 与安全 / 基础

HTTP/1.1 使用消息格式在连接上表达请求与响应，默认允许持久连接。复用减少重复建连开销；流水线请求仍要求按顺序返回响应，前面的慢响应可能阻塞后续响应。

回答要点

- 持久连接意味着一次传输连接可服务多个请求，但不能因此说浏览器和服务器永远保持连接；超时、Connection 语义、资源管理和网络错误都会影响实际连接生命周期。
- 消息边界需按协议处理 Content-Length、分块传输和关闭定界等规则；代理与服务器对边界解析不一致可能产生安全问题，因此不能简单以空行后所有字节都当作正文。
- HTTP/1.1 流水线允许尚未收到前一响应就发送下一请求，但响应仍有顺序约束；实际客户端可能使用多个连接缓解等待，不应把 HTTP/2 的多路复用能力赋予 HTTP/1.1。

常见追问

追问：Keep-Alive 和 TCP keepalive 一样吗？答：不是。HTTP 持久连接用于复用应用请求，TCP keepalive 是传输层存活探测机制。开启探测不能保证业务响应及时，也不能替代应用级请求超时。

易错点

易错点：HTTP/1.1 持久连接不等于同时乱序返回多个响应；也不要吧 HTTP 连接复用、TCP 保活和应用心跳混为同一个功能。

延伸阅读 / 原始参考资料

RFC 9112 • HTTP/1.1

11. HTTP/2 怎样多路复用？还会队头阻塞吗？

HTTP 与安全 / 进阶

HTTP/2 将消息编码为帧，在同一连接上用流标识交错承载多个请求，并使用 HPACK 压缩字段。它缓解应用层顺序响应限制，但 TCP 有序字节流仍可使丢失影响多个流。

回答要点

1. 一个流可包含首部与数据帧，不同流的帧能交错发送，接收方按流标识重新组织消息；多路复用减少为并发请求开多条 TCP 连接的需求，却不代表没有资源限额。
2. HTTP/2 同时具备连接级与流级流量控制，主要约束 DATA 帧；服务端还可限制并发流数量。客户端和服务端需要处理流取消、窗口变化及长连接资源占用。
3. 如果 TCP 丢失一个片段，后续字节即使到达也需等待缺失部分恢复，多个 HTTP/2 流可能共同停顿；因此应用层多路复用并没有自动消除传输层队头阻塞。

常见追问

追问：HTTP/2 必须使用 TLS 吗？答：协议本身定义了不同建立方式，不能从标准层面说只允许 TLS；主流浏览器的部署要求与实际支持通常更严格。回答要区分协议能力和具体客户端的实现策略。

易错点

易错点：认为 HTTP/2 每个流对应独立 TCP 连接，或已经消除所有队头阻塞。流独立性存在于帧组织和应用语义层，底层仍共享连接与网络容量。

延伸阅读 / 原始参考资料

RFC 9113 • HTTP/2

RFC 7541 • HPACK

12. HTTP/3、QUIC 与 UDP 是什么关系？

HTTP 与安全 / 进阶

HTTP/3 在 QUIC 上表达 HTTP 语义，QUIC 在 UDP 上提供可靠多路流、安全和连接管理。各流可靠交付相对独立，减少 TCP 层跨流队头阻塞。

回答要点

1. QUIC 包承载帧和流数据，接收方可独立交付不同流的已到达数据；一个流缺少字节仍要等待该流恢复，但通常不需要把其他流的可用数据一起挡在 TCP 字节序列之后。
2. QUIC 使用 TLS 1.3 完成密钥协商与身份验证，传输参数与安全握手结合；连接标识支持特定条件下路径迁移，而迁移还需要路径验证，并不是可无条件接受任意地址。
3. HTTP/3 使用 QPACK 处理字段压缩，避免照搬 HPACK 的连接顺序依赖；UDP 被阻断时可能无法建立 QUIC，客户端需要考虑基于 TCP 的 HTTP 版本回退，收益应按网络情况测量。

常见追问

追问：HTTP/3 还有队头阻塞吗？答：单个可靠流内部仍受有序交付约束，字段解压也可能因动态表依赖等待；消除的是部分跨流的传输阻塞，拥塞、带宽和应用依赖仍然可能共同限制请求。

易错点

易错点：UDP 不可靠不代表 QUIC 不可靠，也不要说 HTTP/3 在任何网络都更快。握手复用、丢包率、部署环境与回退成本决定实际体验。

延伸阅读 / 原始参考资料

RFC 9114 • HTTP/3

RFC 9000 • QUIC

RFC 9001 • TLS for QUIC

13. HTTP 方法、幂等性和状态码怎么回答？

HTTP 与安全 / 基础

HTTP 方法描述操作语义，状态码描述请求处理结果。安全方法原则上不请求改变服务器状态，幂等方法重复同样请求的预期效果与一次相同；它们不等于响应完全相同或业务零副作用。

回答要点

1. GET 与 HEAD 具有安全语义，PUT 和 DELETE 具有幂等语义，POST 一般不保证幂等；实现必须符合这些约定，但日志与统计等附带行为不改变安全语义的基本定义。
2. 2xx 表示成功，3xx 表示进一步行动或重定向，4xx 多为请求相关问题，5xx 多为服务器无法完成有效请求；细分语义要结合具体码，例如 401 与 403、201 与 204 不能互换。
3. 网络超时可能发生在服务端已经完成操作之后，客户端不知道结果；重试非幂等业务应使用业务幂等键或查询结果等机制，不能只依靠 HTTP 方法名称决定交易是否重复执行。

常见追问

追问：DELETE 第一次返回 204、第二次返回 404 还幂等吗？答：可以。幂等关注预期资源状态的效果，不要求每次响应相同；删除后资源都不再存在，具体错误和响应表达仍由接口契约规定。

易错点

易错点：把幂等理解为任意请求可以安全重试，或认为 GET 参数只能在 URL、POST 天然更安全。协议语义、业务约束和传输安全应分别讨论。

延伸阅读 / 原始参考资料

RFC 9110 • HTTP Semantics

14. HTTP 强缓存与协商缓存如何工作？

HTTP 与安全 / 基础

新鲜缓存可直接复用，过期后可向服务器验证。Cache-Control 管理策略，ETag 等字段充当验证器；304 允许复用本地表示，不携带完整正文。

回答要点

1. max-age 描述响应的新鲜寿命，no-cache 要求复用前验证，no-store 要求不存储；名称不能只按日常含义猜测，还需考虑共享缓存、私有缓存及授权响应的限制。
2. ETag 配合 If-None-Match 按表示标签验证，Last-Modified 配合 If-Modified-Since 按时间验证；弱验证器与强验证器适用场景不同，不能一概用于所有范围请求或并发更新检查。
3. 缓存键不仅可能包括 URL，还可能根据 Vary 区分请求字段；忽略语言、编码或认证相关差异会返回错误版本甚至泄漏内容。更新策略要明确版本化资源与动态数据的不同需求。

常见追问

追问：no-cache 会不会完全不缓存？答：它允许存储，但再次使用前必须按规则验证；no-store 才限制存储。已有缓存的处理、浏览器历史机制和应用存储并非都由一个响应字段直接控制。

易错点

易错点：把 304 当成服务器没有数据，或者认为 ETag 一定是文件 MD5。ETag 是服务器选择的表示验证标签，生成方式属于实现细节。

延伸阅读 / 原始参考资料

RFC 9111 • HTTP Caching

15. HTTPS 与 TLS 1.3 握手如何理解？

HTTP 与安全 / 进阶

HTTPS 通过 TLS 为 HTTP 连接提供机密性、完整性与身份验证。TLS 1.3 通常通过临时密钥交换建立会话密钥，用证书与签名验证服务器，再用对称算法保护后续应用数据。

回答要点

1. 客户端发送支持的算法、密钥份额及扩展，服务端选择参数并提供相应握手信息；证书链、域名、有效期和信任根验证共同决定是否信任服务端，不只是看到证书就算安全。
2. 密钥交换、身份签名与记录加密承担不同职责，不能说证书公钥直接用于加密所有 HTTP 数据；TLS 1.3 移除了旧式 RSA 密钥传输，描述旧 TLS 流程时应注明版本。
3. 会话恢复可以减少握手成本，0-RTT 早期数据存在重放风险，应用需限制可接受的操作；TLS 保护传输链路，但终端被入侵、错误授权或业务逻辑漏洞仍由应用负责处理。

常见追问

追问：HTTPS 能隐藏所有网络信息吗？答：不能。观察者通常仍能看到 IP、连接规模与时间等元数据，传统 DNS 和部分握手信息也可能暴露域名线索；应区分加密内容和可观察的流量特征。

易错点

易错点：把 TLS 1.2 的 RSA 传输流程直接套到 TLS 1.3，或认为 HTTPS 自动保证网站业务可信。握手版本、证书验证与应用安全属于不同层次。

延伸阅读 / 原始参考资料

RFC 8446 • TLS 1.3

16. DNS 解析过程和缓存、UDP/TCP 的关系？

网络基础 / 基础

DNS 把域名映射到资源记录。客户端常向递归解析器查询，后者根据缓存和委派关系访问根、顶级域及权威服务器；缓存受 TTL 约束，DNS 并非只使用 UDP。

回答要点

1. 递归查询要求解析器尽力返回结果，迭代响应可给出下一步应询问的服务器；普通客户端通常不自己遍历所有层级，递归解析器会复用委派和记录缓存减少请求。
2. A、AAAA、CNAME、MX 等记录表达不同信息，域名解析不必只返回一个 IP；负缓存也能保存不存在的结果。TTL 是缓存允许保存的时间参数，不保证所有客户端同时更新。
3. 传统 DNS 常以 UDP 发起查询，响应截断等情况下可改用 TCP，区域传送通常使用 TCP；DoT 和 DoH 还可通过加密传输承载 DNS。讨论连接开销时应明确实际协议。

常见追问

追问：修改 DNS 后为什么有人还访问旧地址？答：递归解析器、系统或应用可能仍持有有效缓存，浏览器和连接池也可能继续使用旧连接。应检查记录 TTL、权威响应、实际解析器和应用连接复用。

易错点

易错点：认为一次查询必然依次访问根和全部权威层级，或者 DNS 永远只用 UDP。缓存、委派、截断回退和加密 DNS 都会改变实际流程。

延伸阅读 / 原始参考资料

RFC 1034 • DNS Concepts and Facilities

RFC 1035 • DNS Implementation

RFC 7766 • DNS over TCP

17. IP 地址、子网掩码与 CIDR 如何计算？

网络基础 / 基础

CIDR 用前缀长度标识网络部分，例如 IPv4 /24 有二十四位前缀和八位主机部分。地址与掩码按位与得到网络地址，是否同网段要结合前缀判断，不能只比较点分十进制外观。

回答要点

1. 例如 192.168.10.34/24 的网络地址为 192.168.10.0，传统该广播子网的广播地址为 192.168.10.255；前缀越长，地址范围越小，路由匹配通常优先选择更长前缀。
2. 常规 IPv4 子网的可用主机数常按 2 的主机位次方减二，但 /31 点到点和 /32 主机路由有特殊语义；回答公式必须指出适用范围，不能对全部前缀机械套用。
3. IPv6 地址长度与 IPv4 不同，也没有 IPv4 式广播机制；网络前缀、路由和地址配置规则要按具体协议说明。NAT、私网地址和子网划分是不同概念，不应混为一谈。

192.168.10.34 & 255.255.255.0 = 192.168.10.0
/24 共 256 个地址；常规广播子网通常可用 254 个

常见追问

追问：两个地址看起来前三段相同就同网段吗？答：不一定，还需要各自配置的前缀长度；/25 会把原 /24 分成两个范围。主机发包时依据路由表决定下一跳，判断不只依赖地址字符串。

易错点

易错点：把 /24 看成二十四台主机，或者对 /31 仍一律减去网络地址和广播地址。先确定地址位宽、前缀与链路语义，再做计算。

延伸阅读 / 原始参考资料

RFC 4632 • CIDR

RFC 3021 • 31-Bit Prefixes

18. 路由、最长前缀匹配和 NAT 的区别？

网络基础 / 基础

路由选择数据包下一跳，最长前缀匹配优先选择最具体的匹配路由；NAT 改写地址或端口实现映射。路由本身不意味着修改端到端地址，NAT 也不是通用安全控制的替代。

回答要点

1. 主机向远端发送时通常把链路帧交给网关，IP 目的仍是远端；路由器按表选择输出接口和下一跳，并递减 TTL，必要时返回 ICMP 错误，避免报文无期限循环。
2. 若同时匹配默认路由和更具体前缀，先选更长前缀；同长度匹配再按系统的优先级、度量或策略处理。策略路由等扩展可能加入源地址和其他条件，应说明普通模型范围。
3. NAT 表记录转换关系，多个内网连接可通过不同端口共用公网地址；这种映射影响入站连接、连接追踪与超时。防火墙过滤规则和 NAT 地址改写应分别理解。

常见追问

追问：为什么内网主动连公网通常容易，公网主动连内网却困难？答：主动连接可以建立 NAT 映射并放行对应返回流量，未知入站连接没有对应目标映射；端口转发或其他连接建立机制可改变这个行为。

易错点

易错点：认为路由器逐跳修改目的 IP 为下一跳 IP，或只要做 NAT 就等于安全。普通转发改变的是链路交付目标，而网络层目的通常仍指向最终目标。

延伸阅读 / 原始参考资料

RFC 1812 • IPv4 Routers

RFC 3022 • Traditional NAT

19. ARP 如何工作？IPv6 用什么？

网络基础 / 基础

ARP 在 IPv4 的本地链路上解析下一跳 IP 对应的链路地址，常先查缓存再广播请求。跨网段访问通常解析网关而非远端主机的 MAC；IPv6 使用邻居发现协议。

回答要点

1. 发送方先按路由确定下一跳，再在本地链路查询该下一跳的地址映射；广播 ARP 请求通常被同链路主机接收，持有目标地址的一方响应，得到映射后才能构造链路帧。
2. ARP 缓存减少每次发送前的广播，条目需要更新和失效；ARP 缺少强认证机制，错误或恶意应答可能改变映射，应结合链路隔离、检查机制和实际部署考虑防护。

3. IPv6 邻居发现基于 ICMPv6，除地址解析外还涉及邻居可达性和路由器发现等功能；不能把 IPv4 ARP 报文直接用于 IPv6，也不能把两者视为报文格式完全相同。

常见追问

追问：访问远程网站需要知道服务器的 MAC 吗？答：一般不需要。主机需要本地下一跳的链路地址，路由器逐跳重新组织链路交付；服务器 MAC 不会作为统一端到端标识穿越整条路由路径。

易错点

易错点：认为 ARP 在互联网范围广播寻找服务器，或 ARP 只查最终目标。它主要服务本地链路交付，目标由路由选择的下一跳决定。

延伸阅读 / 原始参考资料

RFC 826 • Address Resolution Protocol

RFC 4861 • IPv6 Neighbor Discovery

20. 输入 URL 后发生什么？socket 在哪里？

实践与排查 / 基础

浏览器解析 URL、检查策略与缓存，取得目标地址并建立或复用连接，完成必要安全握手后发 HTTP 请求。socket 是程序使用传输服务的接口，页面显示还需要资源加载、解析与渲染。

回答要点

1. TCP 服务端常按 socket、bind、listen、accept 组织连接，客户端使用 connect；accept 返回已连接的通信描述符，监听描述符继续负责接收新连接，两者用途不能混淆。
2. 浏览器可能命中 DNS、HTTP 缓存或连接池，HTTPS 可能使用 TCP+TLS 或 QUIC；重定向会触发新的目标处理，实际链路不必每次从头进行全部步骤，要说明缓存和协议分支。
3. 收到 HTML 后解析 DOM、样式并发起子资源请求，布局、绘制和合成影响可见时间；网络请求完成并不等于页面可交互，排查应分别看 DNS、连接、首字节、下载和渲染阶段。

常见追问

追问：connect 成功能证明服务业务正常吗？答：不能，它主要说明传输连接建立，服务器应用可能繁忙、挂起或返回错误；应该检查业务请求与响应，并设置连接超时和请求超时，避免把探活层次混淆。

易错点

易错点：把 URL 流程背成每次都重新查 DNS 和三次握手，或把 accept 当成创建监听服务。连接复用、HTTP/3 和缓存会改变步骤，接口职责也需明确。

延伸阅读 / 原始参考资料

Linux 手册 • socket(7)

Linux 手册 • accept(2)

